Approximate Solution Methods

Genya Ishigaki

Announcement

Next class:

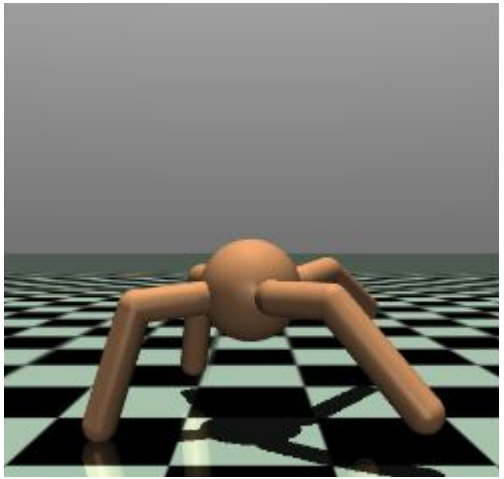
- Implementation of today's content
- Bring your laptop
- You should be able to open ".ipynb" file
- Also, please install
  - numpy
  - Matplotlib (pyplot)

Review: Q-Learning

- There are two actions (Left or Right) from each state
- The current Q table shows
  - Q(S1, L) = 1.0, Q(S1, R) = 0
  - Q(S2, L) = - 1.0, Q(S2, R) = 2.0
  - Q(S3, L) = 5.0, Q(S3, R) = - 1.0
- Assume an agent observed the following experiences
  - S1 L 3 S2
  - S2 R 4 S3

Case Study

- The ant (A 3D robot)
  - One torso (free rotational body) with four legs
  - Each leg having two body parts
- The goal is to coordinate the four legs to move in the forward (right) direction by applying torques on the eight hinges
- Action Space



Action	Min	Max	Name	Joint
Torque applied on the rotor between the back right two links	-1	1	angle_4	hinge

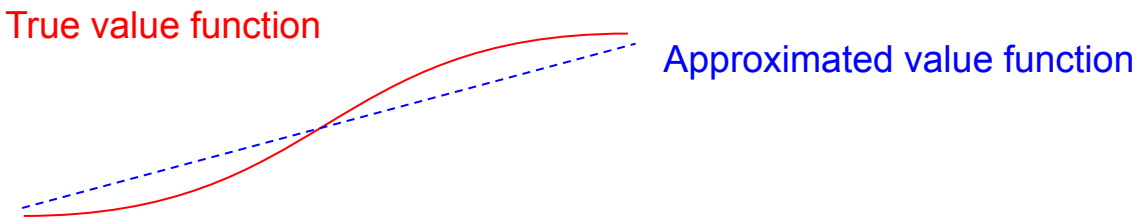
https://gymnasium.farama.org/environments/mujoco/ant/

Generalization

- We want to apply to problems with arbitrarily large state/action spaces
  - More memory needed for large tables
  - Longer time and more data needed to fill the tables accurately
- In many of our target tasks, almost every state/action encountered will never have been seen before
- It is necessary to generalize from previous encounters with different states that are in some sense similar to the current one
- How can experience with a limited subset of the state space be usefully generalized to produce a good approximation over a much larger subset?

Function Approximation

- The kind of generalization we require is often called function approximation
- It takes examples from a desired function (e.g., a value function) and attempts to generalize from them to construct an approximation of the entire function
- Function approximation is an instance of supervised learning



Approximate Value Function

- The approximate value function is represented not as a table but as a parameterized functional form with weight vector

$$\hat{v}(s,\mathbf{w}) \approx v_\pi(s)$$

- We can choose any method to approximate the function
  - Linear function (This lecture): Simple but limited
  - Neural Network (Deep Reinforcement Learning lectures): Complex but potentially more powerful
  - Decision Tree

Linear Approximation: Settings

- Each state should be represented as a feature vector
  - Similar states should be close to each other (in the vector space)

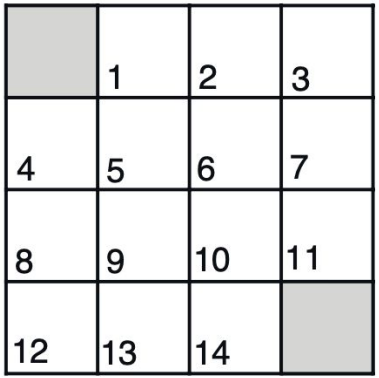
$$\mathbf{x}(S) = [x_1(S), x_2(S), \ldots, x_n(S)]$$

- We assume that the value function can be represented as a linear combination of the features with a weight vector $\mathbf{w} = [w_1, w_2, \ldots, w_n]$

$$\hat{v}(S, \mathbf{w}) = \mathbf{x}(S)^\top \mathbf{w} = x_1(S)w_1 + \cdots + x_n(S)w_n$$

An Example: Grid World

- A state representation: 2D coordinates
- Why?
  - (1,1) and (1,2)
  - (1,1) and (3,3)
- Similarity can be represented based on the encoding



Milestone Questions: State Representation

- Air quality
  - Day1's AQ → S1
  - Day2's AQ → S2
  - …
- Is this a good state representation?
- Can you think of any better way?

Milestone Questions: State Representation

- Air quality
  - Day1's AQ → S1
  - Day2's AQ → S2
  - …
- Is this a good state representation?
- Can you think of any better way?
  - [chemical 1, chemical 2, …]
  - [rainfall, temperature, wind strength, …]

Milestone Questions

- Assume you have two data points
  - X1 = (3, 4), Y1 = 14
  - X2 = (6, 8), Y2 = 28
- Now you want to approximate a linear function underlying the data points.
- What is the size of the weight vector?

Milestone Questions: Benefits of Approximation

- In (tabular) Q-learning, how much data do we need to store?
- In Q-learning with the linear approximation, how much data do we need to store?

Milestone Questions: Benefits of Approximation

- In (tabular) Q-learning, how much data do we need to store?
- In Q-learning with the linear approximation, how much data do we need to store?

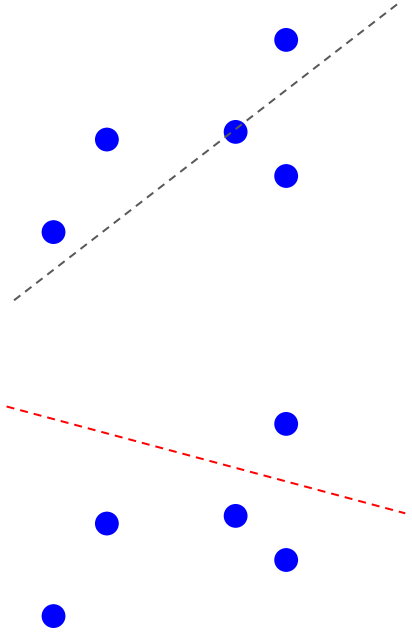
- Q table: |S| x |A|
- A weight vector w: the number of features

Milestone Questions

- Assume you have two data points
  - X1 = (3, 4), Y1 = 14
  - X2 = (6, 8), Y2 = 28
- Now you want to approximate a linear function underlying the data points.
- What will be the best values of the weight?

Finding the best weight parameters

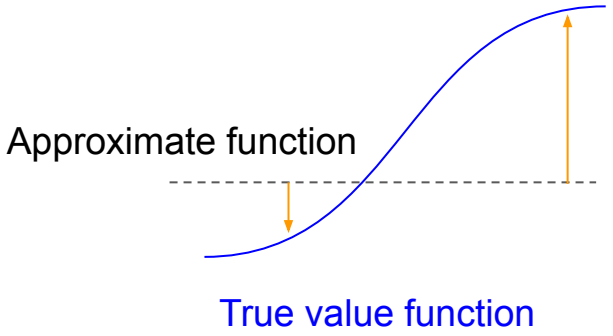
- In general, it's infeasible to find the best weights by random trials
- Define a loss function
  - This function represents how close the approximate function is based on the given data points
- Solve a minimization problem with the loss function



Finding the best weight parameters

- A loss function: Difference between
  - a true value function and
  - an approximate function with the current weight vector w

$$J(\mathbf{w}) \doteq \sum_{s \in \mathcal{S}} \left[v_\pi(s) - \hat{v}(s, \mathbf{w}) \right]^2$$



- In an RL loop, the true value function is not available
- We can approximate the sampled returns many times to obtain the true value function (same thing from the tabular methods)

How do you solve a minimization problem?

- A simple mathematical solution

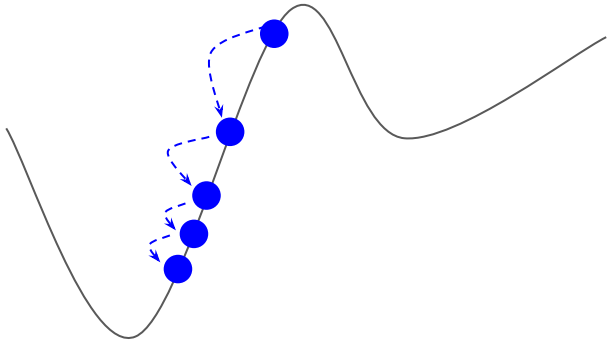
$$J(w) = w^2$$

- Differentiate J with respect to w
- It is not always possible with a complex objective function
- Also, it tends to be computationally expensive

How do you solve a minimization problem?

- Gradient Descent (GD)
  - Starting from an initial point $\mathbf{w}_0$
  - Always find another point $\mathbf{w}_{t+1}$ that the corresponding function value reduces, compared to the one at the previous point $\mathbf{w}_t$

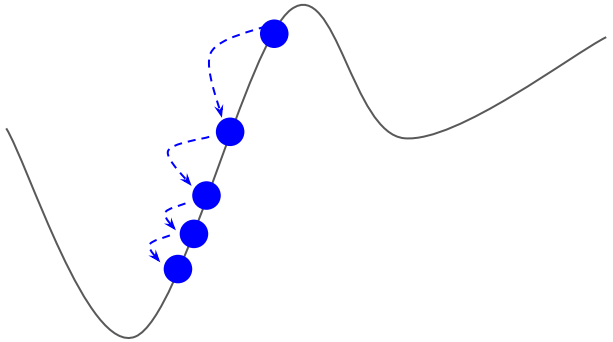
$$J(\mathbf{w}_{t+1}) \leq J(\mathbf{w}_t)$$



How do you solve a minimization problem?

- The gradient $\nabla J(\mathbf{w})$ of a scalar-valued differentiable function J at a point $\mathbf{w}$

$$\nabla J(\mathbf{w}) = \begin{bmatrix} \frac{\partial J}{\partial w_1}(\mathbf{w}) \\ \vdots \\ \frac{\partial J}{\partial w_n}(\mathbf{w}) \end{bmatrix}$$



Batch vs. Stochastic Gradient Descent

- With the actual objective (loss) function

$$J(\mathbf{w}) \doteq \sum_{s \in \mathcal{S}} \left[v_\pi(s) - \hat{v}(s, \mathbf{w}) \right]^2$$

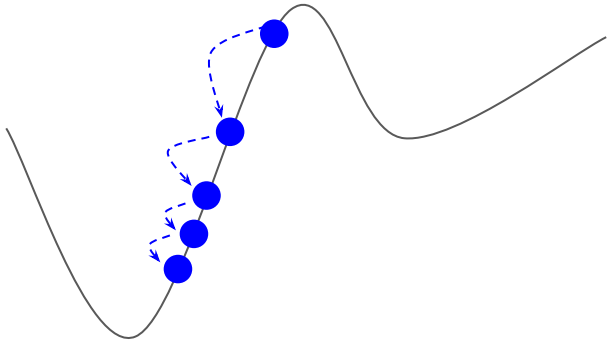
- To update the weight parameter w, you need to have samples for all states
  - This is time consuming to wait for all samples (not even guaranteed)
  - This is computationally inefficient (too many terms)
- We can speed up the GD step by Stochastic GD (SGD)
  - Update the weight parameter by one sample (from one state)
  - Repeat this sample-based update enough times

$$J(\mathbf{w}) \doteq \mathbb{E}\left[\left(v_\pi(s) - \hat{v}(s, \mathbf{w}) \right)^2 \right]$$

How do you solve a minimization problem?

- When dealing with a minimization problem, we take a step along the negative direction of the gradient vector

$$\begin{aligned}\mathbf{w}_{t+1} &\leftarrow \mathbf{w}_t - \frac{1}{2}\alpha\nabla J(\mathbf{w}_t) \\ &= \mathbf{w}_t - \frac{1}{2}\alpha\nabla[v(s) - \hat{v}(s,\mathbf{w}_t)]^2 \\ &= \mathbf{w}_t + \alpha[v(s) - \hat{v}(s,\mathbf{w}_t)]\nabla\hat{v}(s,\mathbf{w}_t)\end{aligned}$$



In practice, the true value function is not available.
Use a sample value (such as a return or a bootstrapped estimate)

Gradient MC (Prediction)

Gradient Monte Carlo Algorithm for Estimating $\hat{v} \approx v_\pi$

Input: the policy π to be evaluated
Input: a differentiable function $\hat{v} : \mathcal{S} \times \mathbb{R}^d \to \mathbb{R}$
Algorithm parameter: step size $\alpha > 0$
Initialize value-function weights $\mathbf{w} \in \mathbb{R}^d$ arbitrarily (e.g., $\mathbf{w} = \mathbf{0}$)

Loop forever (for each episode):
    Generate an episode $S_0, A_0, R_1, S_1, A_1, \ldots, R_T, S_T$ using π
    Loop for each step of episode, $t = 0, 1, \ldots, T-1$:
        $\mathbf{w} \leftarrow \mathbf{w} + \alpha\big[G_t - \hat{v}(S_t,\mathbf{w})\big]\nabla\hat{v}(S_t,\mathbf{w})$

Milestone Questions

- What is the difference between tabular MC and gradient MC?
  - Memory size needed? How scalable?
  - What is updated after each episode?

Target Values by Methods

- Monte Carlo

$$S_t \mapsto G_t$$

- Temporal Difference Learning

$$S_t \mapsto R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w}_t)$$

- Dynamic Programming

$$s \mapsto \mathbb{E}_\pi[R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w}_t) \mid S_t = s]$$

Semi-gradient TD(0) (Prediction)

Semi-gradient TD(0) for estimating $\hat{v} \approx v_\pi$

Input: the policy π to be evaluated
Input: a differentiable function $\hat{v} : \mathcal{S}^+ \times \mathbb{R}^d \to \mathbb{R}$ such that $\hat{v}(\text{terminal},\cdot) = 0$
Algorithm parameter: step size $\alpha > 0$
Initialize value-function weights $\mathbf{w} \in \mathbb{R}^d$ arbitrarily (e.g., $\mathbf{w} = \mathbf{0}$)

Loop for each episode:
  Initialize S
  Loop for each step of episode:
    Choose $A \sim \pi(\cdot|S)$
    Take action A, observe R, S'
    $\mathbf{w} \leftarrow \mathbf{w} + \alpha\big[R + \gamma\hat{v}(S',\mathbf{w}) - \hat{v}(S,\mathbf{w})\big]\nabla\hat{v}(S,\mathbf{w})$
    $S \leftarrow S'$
  until S is terminal

They take into account the effect of changing the weight vector w on the estimate, but ignore its effect on the target. They include only a part of the gradient and, accordingly, we call them semi-gradient methods.

Linear Approximation

- Again, we have so many different functions we can use to estimate the value function
- Let's assume we "chose" a linear function

$$\hat{v}(S, \mathbf{w}) = \mathbf{x}(S)^\top \mathbf{w} = x_1(S)w_1 + \cdots + x_n(S)w_n$$

- What is the gradient of this function?

$$\nabla \hat{v}(S, \mathbf{w}_t) =$$

Linear Approximation

- Again, we have so many different functions we can use to estimate the value function
- Let's assume we "chose" a linear function

$$\hat{v}(S, \mathbf{w}) = \mathbf{x}(S)^\top \mathbf{w} = x_1(S)w_1 + \cdots + x_n(S)w_n$$

- What is the gradient of this function?

$$\nabla \hat{v}(S, \mathbf{w}_t) = \mathbf{x}(S)$$

Milestone Questions

- Assume a 2D grid world where each state is represented as a pair of coordinates
- Assume an agent observed two episodes
  - (1,1) A2 3 (1,2) A1 4 T
- Using first-visit gradient MC Prediction with a linear approximation function, compute the state value
- For simplicity,
  - Initialize w with 0
  - Assume no discounting

Milestone Questions

- Can you come up with (semi-)gradient SARSA?

Milestone Questions

- Can you come up with (semi-)gradient SARSA?

Episodic Semi-gradient Sarsa for Estimating $\hat{q} \approx q_*$

Input: a differentiable action-value function parameterization $\hat{q} : \mathcal{S} \times \mathcal{A} \times \mathbb{R}^d \to \mathbb{R}$
Algorithm parameters: step size $\alpha > 0$, small $\varepsilon > 0$
Initialize value-function weights $\mathbf{w} \in \mathbb{R}^d$ arbitrarily (e.g., $\mathbf{w} = \mathbf{0}$)

Loop for each episode:
  $S, A \leftarrow$ initial state and action of episode (e.g., ε-greedy)
  Loop for each step of episode:
    Take action A, observe R, S'
    If S' is terminal:
      $\mathbf{w} \leftarrow \mathbf{w} + \alpha\big[R - \hat{q}(S, A, \mathbf{w})\big]\nabla\hat{q}(S, A, \mathbf{w})$
      Go to next episode
    Choose A' as a function of $\hat{q}(S', \cdot, \mathbf{w})$ (e.g., ε-greedy)
    $\mathbf{w} \leftarrow \mathbf{w} + \alpha\big[R + \gamma\hat{q}(S', A', \mathbf{w}) - \hat{q}(S, A, \mathbf{w})\big]\nabla\hat{q}(S, A, \mathbf{w})$
    $S \leftarrow S'$
    $A \leftarrow A'$

Milestone Questions: Benefits of Approximation

- Assume an agent experiences an experience (s, a, r, s')

- In (tabular) Q-learning, how much impact does this experience have to the value function? (How many states will be updated?)
- In Q-learning with the linear approximation, how much impact does it have?

Milestone Questions: Benefits of Approximation

- Assume an agent experiences an experience (s, a, r, s')

- In (tabular) Q-learning, how much impact does this experience have to the value function? (How many states will be updated?)
- In Q-learning with the linear approximation, how much impact does it have?

- q(s,a) will be updated (one state-action pair)
- w will be updated. Then, all q values will change (all state-action pairs)

Feature Construction for Linear Methods

- So far, we looked at the weight parameter updates by SGD
- Another key issue is the state representation (feature construction)
  - How should we represent the state?
  - How many features should we use?
- There are many coding options for this task too!
  - E.g. Coarse coding

Coarse coding

- A task in which the natural representation of the states is a continuous two-dimensional space
- Features corresponding to circles in the state space
- Each element of a state vector corresponds to a circle (0 or 1)
- We are going to see an advanced state representation later in this course…

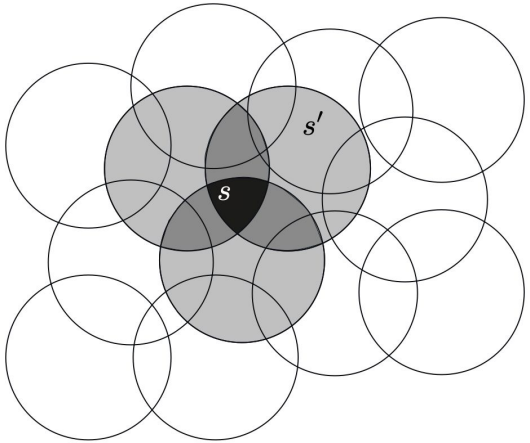


Figure 9.6: Coarse coding. Generalization from state s to state s' depends on the number of their features whose receptive fields (in this case, circles) overlap. These states have one feature in common, so there will be slight generalization between them.

Reference

- If you are not confident about the gradient methods and their mathematical intuition, please make sure you review the following note
- https://openlearninglibrary.mit.edu/assets/courseware/v1/d81d9ec0bd142738b069ce601382fdb7/asset-v1:MITx+6.036+1T2019+type@asset+block/notes_chapter_Gradient_Descent.pdf

Policy Gradient Methods + Approximation Solutions

REINFORCE: Monte Carlo Policy Gradient

REINFORCE: Monte-Carlo Policy-Gradient Control (episodic) for π_*

Input: a differentiable policy parameterization $\pi(a|s,\boldsymbol{\theta})$
Algorithm parameter: step size $\alpha > 0$
Initialize policy parameter $\boldsymbol{\theta} \in \mathbb{R}^{d'}$ (e.g., to $\mathbf{0}$)

Loop forever (for each episode):
  Generate an episode $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$, following $\pi(\cdot|\cdot,\boldsymbol{\theta})$
  Loop for each step of the episode $t = 0, 1, \ldots, T-1$:
    $G \leftarrow \sum_{k=t+1}^{T} \gamma^{k-t-1} R_k$ (G_t)
    $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha\gamma^t G \nabla \ln \pi(A_t|S_t,\boldsymbol{\theta})$

Equivalent to $\frac{\nabla \pi(A_t|S_t,\boldsymbol{\theta}_t)}{\pi(A_t|S_t,\boldsymbol{\theta}_t)}$ (It is called an *eligibility vector*: $\nabla \ln x = \frac{\nabla x}{x}$)

It is an MC method since the parameters are updated based on samples of returns from full episodes

REINFORCE with Baseline

- We subtract a baseline function b(s) from the policy gradient
- Intuition: A smaller value function makes the gradient smaller (and more stable updates)

$$\nabla J(\boldsymbol{\theta}) \propto \sum_s \mu(s) \sum_a \Big(q_\pi(s,a) - b(s) \Big) \nabla \pi(a|s,\boldsymbol{\theta})$$

- The baseline can be any function, even a random variable, as long as it does not vary with action a. Why?

$$\sum_a b(s) \nabla \pi(a|s,\boldsymbol{\theta}) \;=\; b(s) \nabla \sum_a \pi(a|s,\boldsymbol{\theta}) \;=\; b(s) \nabla 1 \;=\; 0$$

- One natural choice for the baseline is an estimate of the state value $\hat{v}(S_t, \mathbf{w})$

REINFORCE with Baseline

REINFORCE with Baseline (episodic), for estimating $\pi_\theta \approx \pi_*$

Input: a differentiable policy parameterization $\pi(a|s,\boldsymbol{\theta})$

Input: a differentiable state-value function parameterization $\hat{v}(s,\mathbf{w})$

Algorithm parameters: step sizes $\alpha^{\boldsymbol{\theta}} > 0$, $\alpha^{\mathbf{w}} > 0$

Initialize policy parameter $\boldsymbol{\theta} \in \mathbb{R}^{d'}$ and state-value weights $\mathbf{w} \in \mathbb{R}^{d}$ (e.g., to $\mathbf{0}$)

Loop forever (for each episode):

Generate an episode $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$, following $\pi(\cdot|\cdot,\boldsymbol{\theta})$

Loop for each step of the episode $t = 0, 1, \ldots, T-1$:

$G \leftarrow \sum_{k=t+1}^{T} \gamma^{k-t-1} R_k$ $\qquad (G_t)$

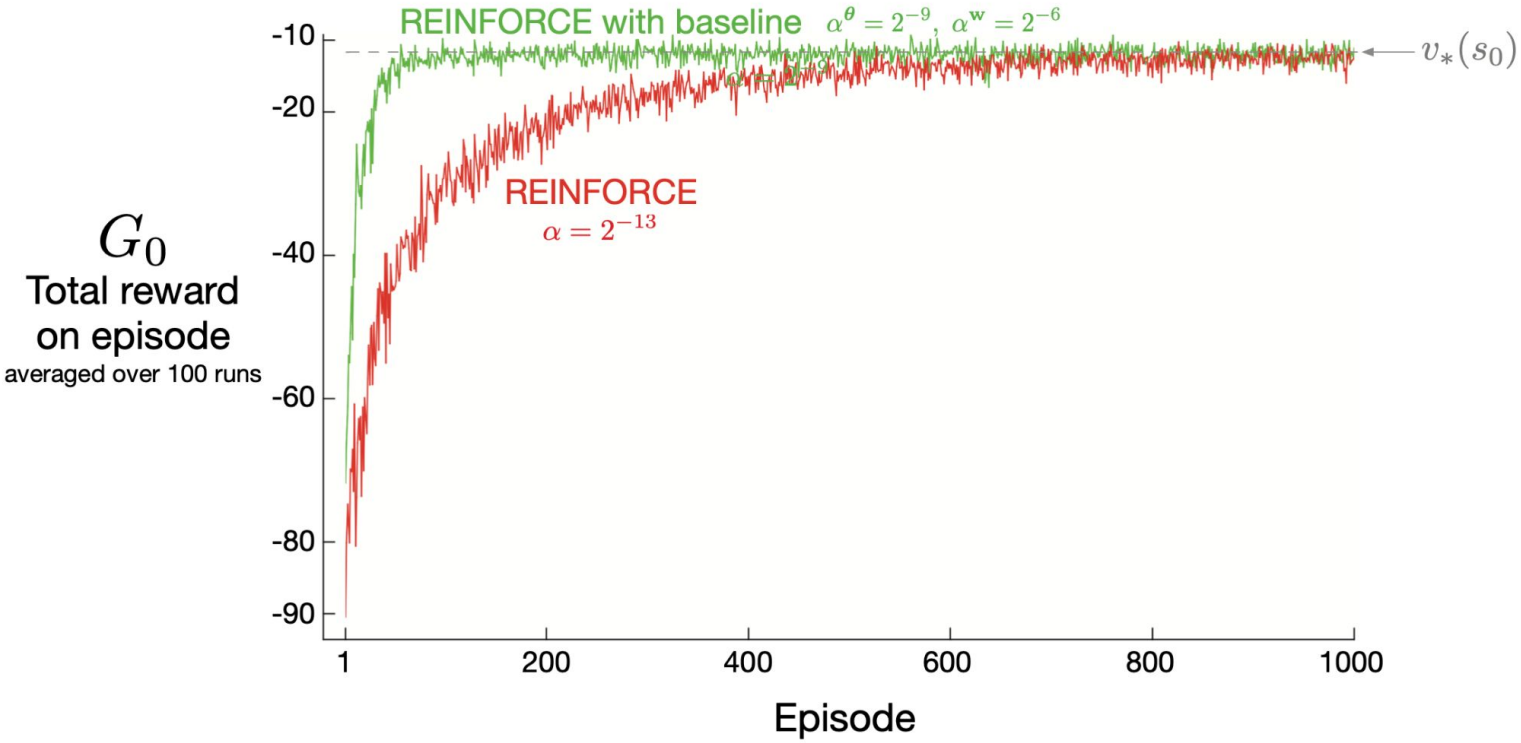
$\delta \leftarrow G - \hat{v}(S_t,\mathbf{w})$

$\mathbf{w} \leftarrow \mathbf{w} + \alpha^{\mathbf{w}} \delta \nabla \hat{v}(S_t,\mathbf{w})$

$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha^{\boldsymbol{\theta}} \gamma^t \delta \nabla \ln \pi(A_t|S_t,\boldsymbol{\theta})$

Remember the MC-based approximation method

REINFORCE on the Short Corridor



See Part 1 for the problem description.

Discussion

- Recap: This is the update equation in REINFORCE (MC PG) with Baseline

$$\boldsymbol{\theta}_{t+1} \doteq \boldsymbol{\theta}_t + \alpha\Big(G_{t:t+1} - \hat{v}(S_t,\mathbf{w})\Big)\frac{\nabla\pi(A_t|S_t,\boldsymbol{\theta}_t)}{\pi(A_t|S_t,\boldsymbol{\theta}_t)}$$

- Even with the baseline, REINFORCE tends to show a higher variance due to the episode sampling
- How can you modify this based on the idea of TD approaches?

Discussion

- Replace the target value!

$$\begin{aligned}\boldsymbol{\theta}_{t+1} &\doteq \boldsymbol{\theta}_t + \alpha\Big(G_{t:t+1} - \hat{v}(S_t,\mathbf{w})\Big)\frac{\nabla\pi(A_t|S_t,\boldsymbol{\theta}_t)}{\pi(A_t|S_t,\boldsymbol{\theta}_t)} \\ &= \boldsymbol{\theta}_t + \alpha\Big(R_{t+1} + \gamma\hat{v}(S_{t+1},\mathbf{w}) - \hat{v}(S_t,\mathbf{w})\Big)\frac{\nabla\pi(A_t|S_t,\boldsymbol{\theta}_t)}{\pi(A_t|S_t,\boldsymbol{\theta}_t)} \\ &= \boldsymbol{\theta}_t + \alpha\delta_t\frac{\nabla\pi(A_t|S_t,\boldsymbol{\theta}_t)}{\pi(A_t|S_t,\boldsymbol{\theta}_t)}.\end{aligned}$$

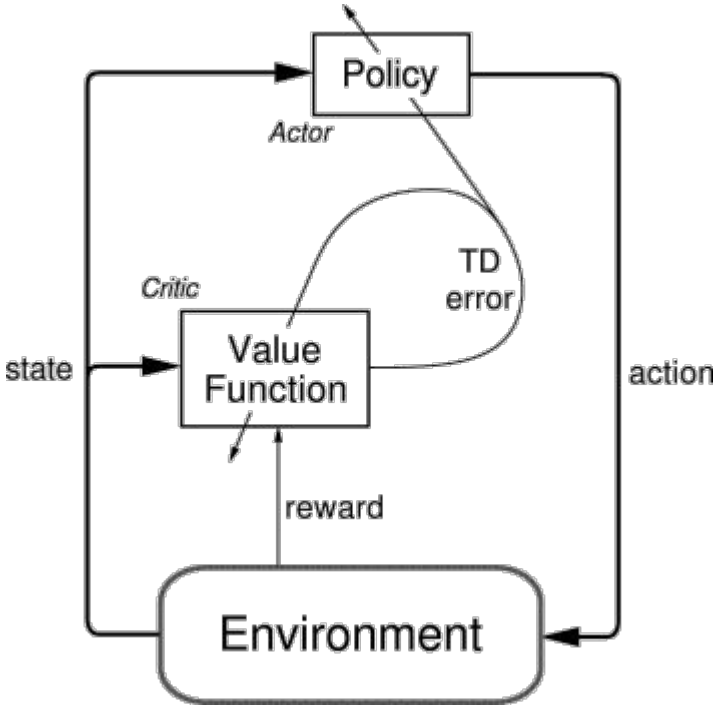
Actor-Critic Methods

One-step Actor–Critic (episodic), for estimating $\pi_\theta \approx \pi_*$

Input: a differentiable policy parameterization $\pi(a|s,\boldsymbol{\theta})$
Input: a differentiable state-value function parameterization $\hat{v}(s,\mathbf{w})$
Parameters: step sizes $\alpha^{\boldsymbol{\theta}} > 0$, $\alpha^{\mathbf{w}} > 0$
Initialize policy parameter $\boldsymbol{\theta} \in \mathbb{R}^{d'}$ and state-value weights $\mathbf{w} \in \mathbb{R}^{d}$ (e.g., to $\mathbf{0}$)
Loop forever (for each episode):
  Initialize S (first state of episode)
  $I \leftarrow 1$
  Loop while S is not terminal (for each time step):
    $A \sim \pi(\cdot|S,\boldsymbol{\theta})$
    Take action A, observe S', R
    $\delta \leftarrow R + \gamma \hat{v}(S',\mathbf{w}) - \hat{v}(S,\mathbf{w})$ (if S' is terminal, then $\hat{v}(S',\mathbf{w}) \doteq 0$)
    $\mathbf{w} \leftarrow \mathbf{w} + \alpha^{\mathbf{w}} \delta \nabla \hat{v}(S,\mathbf{w})$
    $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha^{\boldsymbol{\theta}} I \delta \nabla \ln \pi(A|S,\boldsymbol{\theta})$
    $I \leftarrow \gamma I$
    $S \leftarrow S'$

Actor-Critic Methods

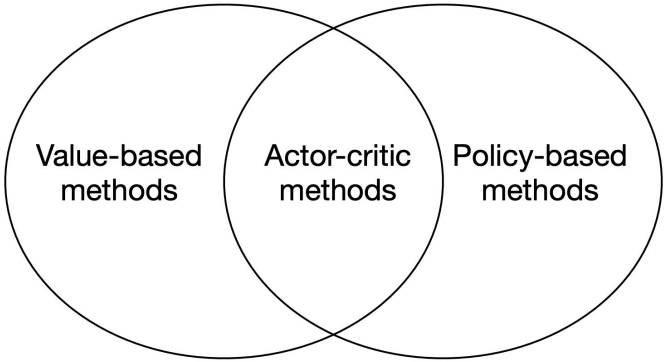
- You have two separate entity:
  - Actor: Updates the policy function (parameters θ)
  - Critic: Updates the action-value function (parameters w)
- The action values estimated by the critic influence the policy update
- The bootstrapped one-step return is often superior to the actual return in terms of its variance

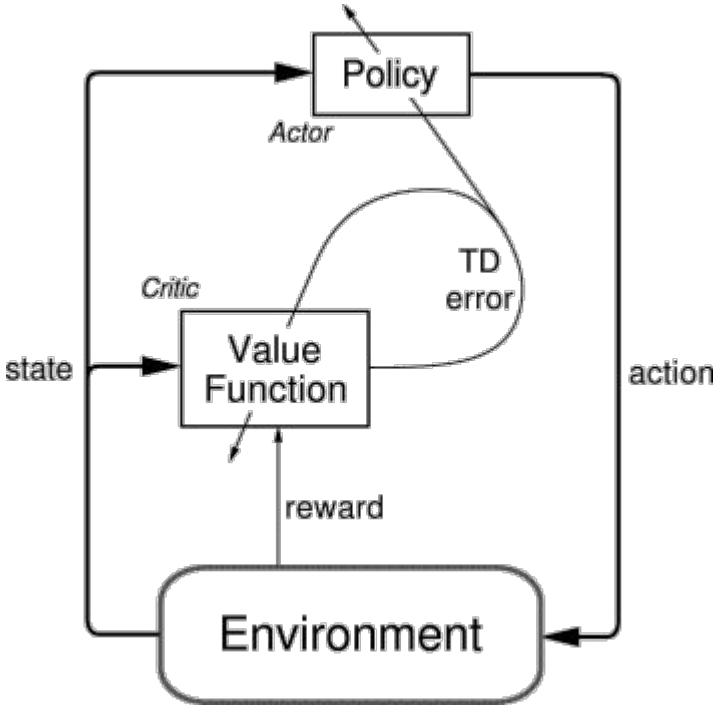


http://incompleteideas.net/book/ebook/node66.html

Actor-Critic Methods

- Actor: A policy-based method for the policy improvement
- Critic: A value-based method for the estimation of state values





The Advantage Function

- What does it mean to use the TD error when updating the policy?

$$\delta_{\pi_\theta} = r + \gamma V_{\pi_\theta}(s') - V_{\pi_\theta}(s)$$

- This is similar to the baseline approach
- We compute the relative impact of taking action a based on the average performance at state s

$$\begin{aligned}\mathbb{E}_{\pi_\theta}[\delta_{\pi_\theta} \mid s, a] &= \mathbb{E}_{\pi_\theta}\left[r + \gamma V_{\pi_\theta}(s') \mid s, a\right] - V_{\pi_\theta}(s) \\ &= Q_{\pi_\theta}(s, a) - V_{\pi_\theta}(s) \\ &= A_{\pi_\theta}(s, a)\end{aligned}$$